Loops in Python

Complete reference: for, while, range, enumerate, zip, control flow and comprehensions

for and while at a glance

```
for item in iterable:       # known items / count
    ...
while condition:            # unknown count
    ...
    # update something!
else:                       # optional: no break
    ...
```

range() cheats

```
range(5)          # 0 1 2 3 4
range(2, 6)       # 2 3 4 5
range(0, 10, 3)   # 0 3 6 9
range(5, 0, -1)   # 5 4 3 2 1
```

Pick the right tool

I want to...	Use
visit every item	`for x in items`
need the position too	`for i, x in enumerate(items)`
walk two lists together	`for a, b in zip(xs, ys)`
loop in reverse	`for x in reversed(items)`
keys and values of a dict	`for k, v in d.items()`
repeat until a condition	`while cond:`
loop forever until `break`	`while True:`
build a list from a loop	`[f(x) for x in xs if cond]`

Loop control

Keyword	Does
`break`	exit the loop immediately
`continue`	jump to the next iteration
`else`	runs if no `break` occurred
`pass`	empty placeholder body

Common patterns

```
total = 0
for n in [4, 8, 15]:
    total += n                      # accumulate
squares = [n * n for n in range(4)]  # transform
big = [n for n in [4, 8, 15] if n > 5]  # filter
counts = {}
for ch in "banana":
    counts[ch] = counts.get(ch, 0) + 1   # count
print(total, squares, big)
print(counts)
```

OUTPUT

```
27 [0, 1, 4, 9] [8, 15]
{'b': 1, 'a': 3, 'n': 2}
```

Dictionaries, strings, files

```
d = {"a": 1, "b": 2}
for k in d: ...                 # keys
for v in d.values(): ...        # values
for k, v in d.items(): ...      # both
for ch in "hey": ...            # characters
for line in open("f.txt"): ...  # lines
```

Comprehension family

```
[x * 2 for x in range(3)]       # → [0, 2, 4]
{x: x * x for x in range(3)}    # → {0: 0, 1: 1, 2: 4}
{x % 3 for x in range(6)}       # → {0, 1, 2}
sum(x * x for x in range(4))    # → 14
```

Nested loops and itertools

```
from itertools import product, islice
for a, b in product("ab", [1, 2]):
    print(a, b, end=" | ")
print(list(islice(range(100), 3)))
```

OUTPUT

```
a 1 | a 2 | b 1 | b 2 | [0, 1, 2]
```

Pitfalls checklist

Pitfall	Fix
changing a list while looping	build a new list
off-by-one with `range`	stop is excluded
infinite `while`	update the condition
reusing an exhausted iterator	wrap in `list()`
`range(len(x))` everywhere	`enumerate(x)`

REMEMBER

- `for` for known collections, `while` for open-ended conditions.
- `enumerate` for the index, `zip` for parallel lists, `.items()` for dict pairs.
- `break` stops, `continue` skips, loop `else` means "no break".
- Comprehensions are the idiomatic way to transform or filter.
- Never modify a collection while iterating over it.
- `:=` can capture a value inside a loop test: `while chunk := f.read(n)`.